

NanoPQ-SecureLink: Memory-Bounded Post-Quantum Authorization and Authenticated Encryption for 8-Bit Microcontrollers

Jeyashree Narayanan, Vishwa, Saravanan, Naveen

Abstract

Deploying post-quantum cryptography on legacy 8-bit microcontrollers remains difficult because standardized lattice-based key-encapsulation mechanisms exceed the memory budgets of devices such as the ATmega328P. The supplied benchmark evidence shows that a previous whole-session ML-KEM-512 implementation requires at least 2,627 bytes of peak SRAM on a device with only 2,048 bytes available, making direct execution infeasible. NanoPQ-SecureLink addresses this constraint by changing the role of public-key cryptography: instead of performing a post-quantum key encapsulation during every session, the device streams a post-quantum authorization signature into a constant-memory verifier during enrollment or policy cutover, persists the authorization result and monotonic policy state in EEPROM, and derives per-session traffic keys from a provisioned 256-bit device root, a boot epoch, and peer nonces using HKDF. Short application records are then protected with Ascon-AEAD128. Across the supplied AVR results, the LMS H5/W4 profile achieves a 1,354-byte peak SRAM footprint with 694 bytes of headroom and 18,188 bytes of flash, while the stateless SLH-DSA profile achieves 1,364 bytes of peak SRAM with 684 bytes of headroom. The resulting composition demonstrates a constrained-device design that provides post-quantum authorization and quantum-resistant symmetric traffic protection within the limits of a 2 KB SRAM ATmega328P, while explicitly not claiming on-device public-key key exchange or public-key forward secrecy.

Index Terms - post-quantum cryptography, embedded security, ATmega328P, LMS, SLH-DSA, Ascon, HKDF, IoT, lightweight cryptography

I. Introduction

Post-quantum migration for constrained embedded systems is harder than simply replacing one primitive with another. In the supplied NanoPQ materials, the core problem is concrete: a previous whole-session ML-KEM-512 design for the ATmega328P requires 2,016 bytes of static SRAM plus a compiler-reported 611-byte connect() frame, yielding at least 2,627 bytes of peak SRAM. That exceeds the physical 2,048-byte SRAM capacity of the target microcontroller and makes direct execution infeasible.

NanoPQ-SecureLink responds by changing the frequency and role of public-key operations. A post-quantum signature is used rarely, during initial enrollment or policy cutover, to authorize a fixed 48-byte policy manifest. The device verifies that signature in a streaming, constant-memory manner, journals the authorization outcome and policy sequence in EEPROM, and then performs routine traffic protection using derived symmetric state and Ascon-AEAD128.

The main contribution is therefore practical and system-level rather than based on a new hardness assumption. The work shows that an 8-bit AVR microcontroller with 2 KB SRAM can participate in a post-quantum-authorized secure channel if the design avoids whole-session on-device KEM execution and instead uses streamed hash-signature verification plus symmetric derivation from an independently provisioned secret.

II. Background and Design Rationale

The uploaded source set positions ML-KEM-512 as the rejected baseline for this exact AVR target, not because ML-KEM is cryptographically unsuitable, but because the current implementation does not fit

the ATmega328P memory envelope. The benchmark table marks the ML-KEM-512 profile as not runnable and failed under link or runtime bounds, while all replacement authorization profiles complete a linked AVR image and pass simulation.

The default authorization profile is LMS H5/W4 as specified by RFC 8554, with a 56-byte public key and a 2,348-byte signature. An alternative LMS H5/W8 profile reduces the signature size to 1,292 bytes, but measured AVR execution reverses the expected trade-off: W8 is much slower because its maximum Winternitz chain length is 255 rather than 15, leading to substantially higher verifier work. A stateless alternative, SLH-DSA-SHA2-128s from FIPS 205, avoids signer-state management at the cost of a larger 7,856-byte signature and longer measured authorization latency.

For traffic protection, NanoPQ-SecureLink uses standardized components rather than custom primitives. Session keys are derived with HKDF over a device root, transcript, boot epoch, and peer nonce, and records are protected with Ascon-AEAD128. This composition grounds the protocol in established standards while reallocating scarce SRAM away from whole-session public-key exchange.

III. System Model and Threat Model

NanoPQ-SecureLink assumes a low-end ATmega328P leaf device, an external peer that stores the matching device root confidentially, and a trusted provisioning path for the verification key and root material. Under the LMS profile, the leaf device is a verifier rather than a signer. It maintains a persistent boot epoch and policy state in EEPROM and enforces exact sequence monotonicity in each session.

Within the demonstrated threat model, the design aims to protect against passive capture of live records, modification of a record or its authenticated header, replay within a session, reuse of the previous session after a normal reset, unauthorized initial policy enrollment without a valid LMS or SLH-DSA signature, UART overrun during expensive signature verification through verifier-driven flow control, and accidental or torn EEPROM journal writes.

The work is also explicit about what it does not provide. It does not claim public-key key agreement or KEM semantics, public-key forward secrecy, recovery after compromise of the 256-bit device root, resistance to invasive physical extraction or arbitrary flash or EEPROM rewriting, side-channel certification, availability under radio or UART jamming, reordering windows beyond exact-next-sequence acceptance, or unlimited EEPROM endurance. These boundaries are central to presenting the work honestly.

IV. Protocol Overview

The protocol begins with a fixed 48-byte signed policy manifest containing a magic value, manifest version, KDF suite, traffic suite, authorization mode, device-class identifier, legacy-policy digest, monotonic policy sequence, and reserved bytes. The device checks manifest structure, device class, policy digest, and increasing sequence before accepting a signature. Signature bytes are requested only when the verifier is ready, preventing AVR UART overflow while hash computations are in progress.

Under the LMS H5/W4 profile, the device verifies a 2,348-byte signature while keeping only one 32-byte chain value and one 32-byte authentication-path node live at a time. The H5/W8 variant retains the same verifier structure with a shorter signature, and the SLH-DSA-SHA2-128s option offers a stateless authorization alternative. In all cases, the design goal is the same: the MCU verifies post-quantum authority tokens without buffering whole signatures in SRAM.

For session establishment, the device computes a deterministic client nonce from the 32-byte device root, an 8-byte device identifier, and a persistent non-zero boot epoch. It sends an authenticated HELLO carrying the device identifier, epoch, and client nonce. The peer rejects any epoch that is not greater than its last accepted epoch and replies with an authenticated challenge containing the epoch,

client nonce, and a 16-byte server nonce. A transcript hash becomes HKDF salt and expands into per-direction send chains, nonce bases, and finished keys. Both sides then exchange 16-byte Finished tags before application data flows.

Application records use independent per-direction chains and 32-bit sequence numbers. For sequence number i , a message key is derived from the current chain, the chain is advanced, and the 16-byte Ascon nonce is formed by XORing i into the last four bytes of a direction-specific nonce base. The receive chain advances only after successful tag verification. In the supplied design, frame sizes remain tightly bounded, including 48-byte HELLO, 60-byte CHALLENGE, 24-byte FINISHED, 29 bytes of data overhead, and 24 bytes maximum plaintext per data frame.

Reset safety is handled through two 18-byte EEPROM journal slots containing schema, authorization flag, generation counter, boot epoch, policy sequence, and CRC-16. The inactive slot is written with magic cleared, fields are updated, and magic is written last. On boot, the newest valid record is selected, the epoch is incremented and persisted before the session begins, and epoch exhaustion is fail-closed. The CRC is used for torn-write detection rather than cryptographic integrity.

V. Implementation on ATmega328P

The supplied benchmark snapshot targets an ATmega328P at 16 MHz with 32 KiB flash and 2,048 bytes SRAM, using a bundled AVR GCC from `quirkbot-avr-gcc 2.0.4` and `avr8js 0.21.0` as simulator. Acceptance gates require total peak SRAM at or below 1,792 bytes, measured SRAM headroom of at least 256 bytes, no signature buffer in SRAM, and successful execution of full enrollment, handshake, encrypted traffic, tamper, replay, and reset paths. Physical Nano validation is explicitly treated as a separate required gate.

The reproduction flow is equally explicit. Physical evaluation uses `make verify`, `make benchmark`, and `make peer`, with `make flash` selecting LMS H5/W4 by default. The host-side peer is expected to demonstrate streamed signature verification without SRAM buffering, mutual HKDF-based session establishment, successful encrypted telemetry, tamper rejection, valid command acceptance, replay rejection, and correct reset behavior. The documentation also warns reviewers not to relabel simulator timing or SRAM usage as physical-board measurements.

The evidence package further states that the firmware performs no dynamic allocation, that sanitizer coverage is applied to host implementations and streaming verifiers, and that an independent LMS check against Cisco's hash-sigs reference implementation accepts both W4 and W8 vectors while rejecting tampered inputs. This strengthens confidence that the LMS verification path is not merely internally self-consistent but independently cross-checked.

VI. Experimental Methodology

The uploaded evidence defines six validation layers. Host tests validate SHA-256, HMAC, Ascon vectors, protocol agreement, tamper handling, replay handling, and reset freshness. Sanitizer tests run address and undefined-behavior sanitizers over host implementations and streaming verifiers. AVR cross-link analysis extracts flash, static SRAM, EEPROM, section summaries, disassembly, and compiler frame usage. AVR execution in `avr8js` validates the full compiled enrollment, session, traffic, and reset path. Gate aggregation rebuilds a machine-readable benchmark table and fails automatically if a replacement profile exceeds 1,792 bytes peak SRAM or drops below 256 bytes headroom. Finally, an independent LMS reference test checks W4 and W8 vectors against Cisco's RFC 8554 implementation.

Peak SRAM accounting is also carefully defined. For the replacement profiles, peak stack is taken as the maximum of direct minimum-stack-pointer tracking and a 0xA5 SRAM watermark; peak SRAM is then computed as `linked .data` plus `.bss` plus `.noinit` plus measured executed stack. For the rejected ML-KEM-512 baseline, the reported 2,627-byte figure is explicitly presented as a conservative lower

bound based on static memory plus the largest compiler-reported connect() frame, excluding interrupts, transport overhead, and deeper nesting. This distinction keeps the comparison technically honest.

VII. Results

The same-target benchmark results are the strongest quantitative evidence in the package. ML-KEM-512 is reported at 20,118 bytes of flash, 2,016 bytes of static SRAM, executed stack of at least 611 bytes, and peak SRAM of at least 2,627 bytes, leaving at most negative 579 bytes of headroom. It is therefore not runnable on the ATmega328P. By contrast, LMS H5/W4 uses 18,188 bytes of flash, 587 bytes of static SRAM, 767 bytes of executed stack, and 1,354 bytes of peak SRAM, leaving 694 bytes of headroom and passing AVR simulation. LMS H5/W8 shows the same memory footprint but requires substantially longer authorization time. SLH-DSA-SHA2-128s uses 19,268 bytes of flash, 597 bytes of static SRAM, 767 bytes of executed stack, and 1,364 bytes of peak SRAM, leaving 684 bytes of headroom.

An especially instructive result is the W4 versus W8 comparison. W8 initially appears attractive because its signature is 1,056 bytes shorter, but measured AVR execution shows the opposite practical outcome: W4 requires 105,538,287 cycles, whereas W8 requires 753,474,158 cycles, making W8 approximately 7.14 times slower. The supplied explanation attributes this reversal to the maximum Winternitz chain length, which is 15 for W4 and 255 for W8. On a low-end MCU, minimizing network artifact size does not necessarily minimize end-to-end execution cost.

The physical Nano runbook provides expected milestones for hardware confirmation rather than overclaiming fully captured physical measurements in the same artifact set. Expected console output includes streamed LMS enrollment, verification without SRAM buffering, HKDF/HMAC session establishment, encrypted traffic, tamper rejection, valid command acceptance, and replay rejection. The evidence labeling also separates simulator success from the requirement for a real-board run.

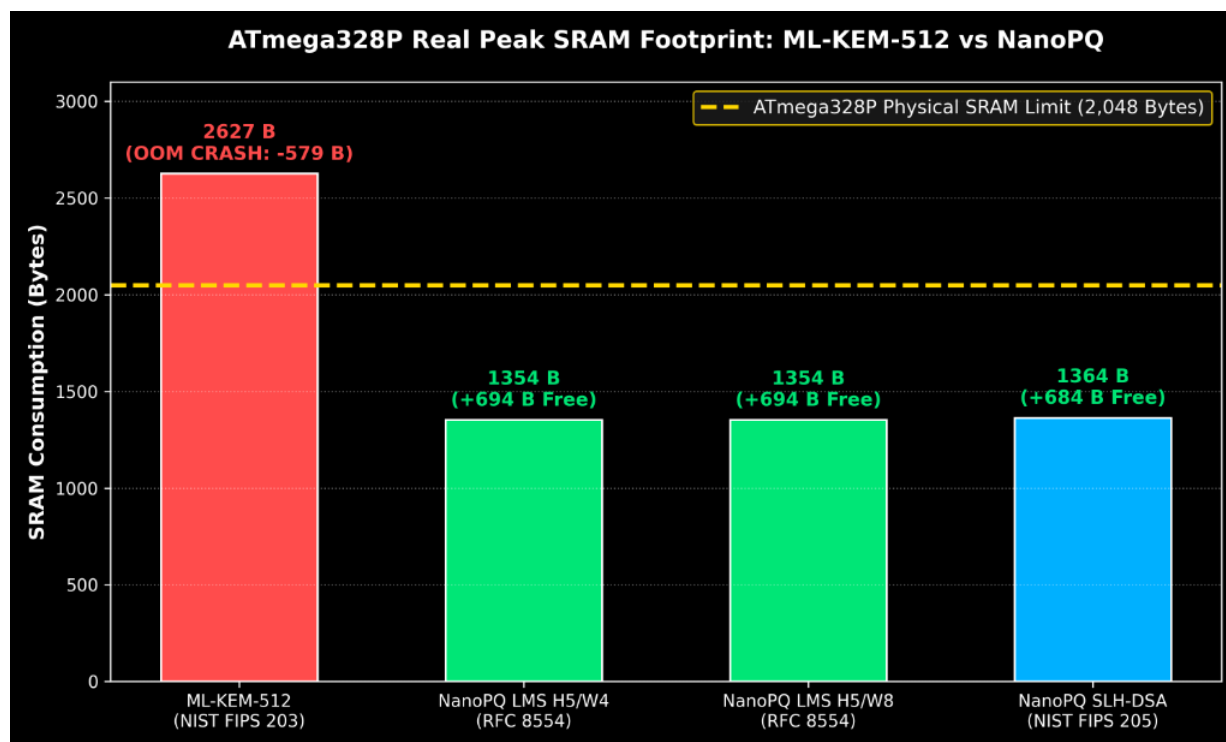


Figure 1. Reported peak SRAM footprint comparison for ML-KEM-512 and NanoPQ authorization profiles on the ATmega328P. Figure reproduced from the user-supplied benchmark artifact.

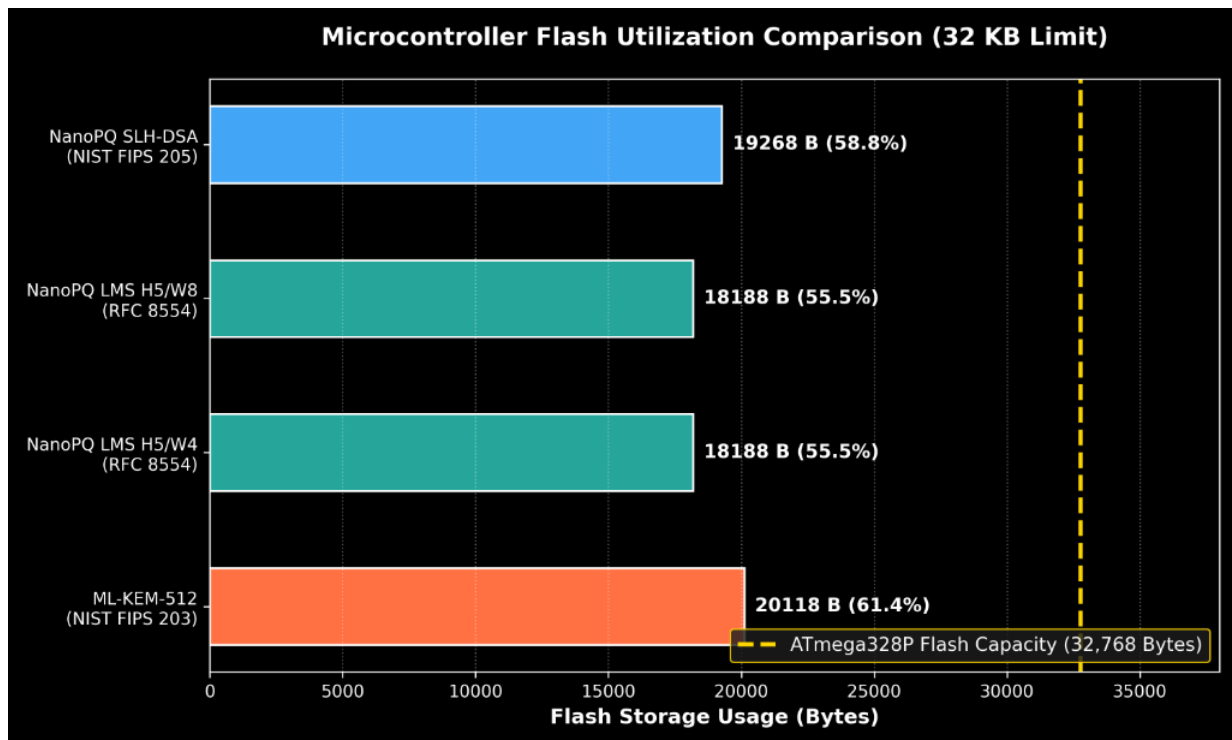


Figure 2. Reported flash utilization comparison across the evaluated profiles under the 32 KB ATmega328P flash limit. Figure reproduced from the user-supplied benchmark artifact.

VIII. Security Scope, Limitations, and Trade-Offs

The protocol's strongest security claim is not that it realizes a post-quantum KEM on an 8-bit AVR, but that it achieves post-quantum authorization plus quantum-resistant symmetric traffic protection from a high-entropy 256-bit device root. The materials are explicit that the construction is not a KEM, does not provide public-key forward secrecy, and should not be substituted for zero-touch public-key onboarding when those semantics are required.

Several trade-offs are unavoidable. First, authorization latency is high on a 16 MHz AVR: 6.596 seconds for LMS H5/W4 and 23.638 seconds for SLH-DSA-SHA2-128s in the supplied measurements. Second, signature payloads are much larger than classical embedded signatures: 2,348 bytes for LMS H5/W4 and 7,856 bytes for SLH-DSA-SHA2-128s. Third, under LMS H5 the signer's tree capacity is limited and must be managed safely, although the architecture reduces this burden by making the MCU a verifier only. Fourth, replacing ephemeral KEM exchange with a pre-shared root plus ratcheting means that exposure of the root is a serious event. Fifth, the ATmega328P remains susceptible to physical side-channel and fault-injection attacks outside the remote threat model.

The materials also warn against misinterpretation. A successful simulator run is not an energy measurement, a valid test vector is not certification, and a signature-authorized pre-shared-key architecture should not be described as a public-key KEM. Preserving these cautions materially improves the credibility of the work.

IX. Reproducibility and Artifact Availability

The benchmark artifact set identifies both the commands and the evidence files required for reproduction. Rebuilding begins with `npm ci --cache .npm-cache` and `make benchmark`, while deeper validation uses `make host-test`, `make sanitizer-test`, `make evidence`, `make simulate`, and optionally `make reference-lms` with an external hash-sigs reference tree. The benchmark package points reviewers to machine-readable end-to-end evidence, size summaries, section summaries, stack-frame summaries, disassembly, and the same-target benchmark CSV.

The physical verification runbook adds the required board-side procedure: optional factory reset to clear the two EEPROM journal slots, flashing the default LMS H5/W4 image, running the peer, and confirming enrollment, traffic protection, tamper rejection, replay rejection, and reset behavior. It also specifies what evidence a reviewer should capture, including board and bootloader details, compiler version, avr-size output, peer console logs, demonstration video, physical runtime, and, if available, supply and SRAM measurements.

X. Conclusion

NanoPQ-SecureLink demonstrates a credible path for bringing post-quantum authorization to an ATmega328P-class 8-bit microcontroller without claiming capabilities the platform cannot support. The core result is architectural: if post-quantum public-key operations are restricted to rare streamed signature verification, and routine traffic is protected by derived symmetric state using standardized primitives, the full protocol path fits within a 2 KB SRAM AVR target.

In the supplied measurements, the default LMS H5/W4 profile reduces peak SRAM from the ML-KEM baseline's at-least 2,627 bytes to 1,354 bytes and leaves 694 bytes of headroom, while also remaining within the 32 KB flash budget. Just as importantly, the materials define the system's limits clearly. The design does not realize on-device public-key key exchange, does not claim public-key forward secrecy, and does not solve physical extraction or certified side-channel resistance on commodity 8-bit silicon. Within those stated limits, however, the work provides a technically honest and experimentally supported demonstration of memory-bounded post-quantum authorization and authenticated encrypted traffic for legacy microcontrollers.

References

- [1] NIST, 'Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203).' <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf>
- [2] D. McGrew, M. Curcio, and S. Fluhrer, 'Leighton-Micali Hash-Based Signatures (RFC 8554).' <https://www.rfc-editor.org/rfc/rfc8554>
- [3] NIST, 'Recommendation for Stateful Hash-Based Signature Schemes (SP 800-208).' <https://csrc.nist.gov/pubs/sp/800/208/final>
- [4] NIST, 'Stateless Hash-Based Digital Signature Standard (FIPS 205).' <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.205.pdf>
- [5] NIST, 'Ascon-Based Lightweight Cryptography Standards for Constrained Devices (SP 800-232).' <https://csrc.nist.gov/pubs/sp/800/232/final>
- [6] H. Krawczyk and P. Eronen, 'HMAC-based Extract-and-Expand Key Derivation Function (HKDF), RFC 5869.' <https://www.rfc-editor.org/rfc/rfc5869>
- [7] Cisco, 'hash-sigs: RFC 8554 reference implementation.' <https://github.com/cisco/hash-sigs>

Note: This PDF was typeset from the user-supplied manuscript content and benchmark figures. Numerical results, security boundaries, and algorithm selections are preserved from the provided source material.